

C# — Conceptos

Germán Gibbs

12 de septiembre de 2026

1. Definiciones

C# es un lenguaje de programación creado por Microsoft que se ejecuta sobre la plataforma .NET.

Es un lenguaje compilado, lo que significa que el código fuente se transforma antes de su ejecución. También es un lenguaje orientado a objetos, por lo que permite organizar la lógica de los programas mediante clases y objetos.

Además, C# es fuertemente tipado: las variables están asociadas a tipos de datos específicos. Al formar parte del ecosistema .NET, permite desarrollar aplicaciones web, videojuegos, servicios en la nube, aplicaciones móviles, aplicaciones de escritorio y soluciones relacionadas con Machine Learning.

Su sintaxis y diseño recibieron influencia de lenguajes como C++, Java y otros lenguajes de la familia C.

Anders Hejlsberg — 2000, Microsoft (open source en 2014)

Al C# ser un lenguaje de alto nivel, el CPU no entiende C Sharp de forma nativa, para que esto suceda tiene que darse un flujo o proceso de compilación que trabaja en conjunto con la plataforma .NET.

C# → Common Intermediate Language (CIL) - → Common Language Runtime (CLR) - Código Binario

1. Código C# - App.cs
2. Código Intermedio - App.exe o App.dll
3. Código Binario - Compilador JIT

2. Características de C#

Característica	Característica	Característica
Simple	Multiparadigma	Interoperable
Orientado a objetos	Estructurado	Fuertemente tipado
Funciones	Componentes	Colecciones

3. Herramientas de desarrollo

Entre las principales herramientas para desarrollar con C# se encuentran:

- Visual Studio
- Visual Studio Code
- JetBrains Rider
- .NET Fiddle

4. ¿Por que C Sharp (C#)

1. Moderno
2. Alta Demanada
3. Multiple Desarrollos

5. Variables y Tipos de Datos

Una variable es un espacio de almacenamiento con un nombre asociado a un tipo de dato específico. En C#, ese tipo se fija en tiempo de compilación y no puede cambiar durante la ejecución del programa (lenguaje fuertemente tipado y de tipado estático).

```
int edad = 20;  
double precio = 19.99;  
char inicial = 'G';  
bool esValido = true;  
string nombre = "German";
```

Cada palabra clave de tipo en C# (int, double, etc.) es en realidad un alias de un tipo definido en la biblioteca base de .NET (System). Por ejemplo, int y System.Int32 son el mismo tipo y pueden usarse indistintamente.

5.1 Tipos de valor incorporados

Palabra clave	Tipo .NET	Categoría
bool	System.Boolean	Lógico
char	System.Char	Carácter
byte / sbyte	System.Byte / System.SByte	Entero
short / ushort	System.Int16 / System.UInt16	Entero
int / uint	System.Int32 / System.UInt32	Entero
long / ulong	System.Int64 / System.UInt64	Entero
float	System.Single	Punto flotante
double	System.Double	Punto flotante
decimal	System.Decimal	Punto flotante de alta precisión

A diferencia de los anteriores, object y string son tipos de **referencia**, no de valor. Se explican en la siguiente sección.

5.2 Inferencia de tipo con var

```
var edad = 20;           // el compilador infiere int  
var nombre = "Ana";     // el compilador infiere string
```

Es importante aclarar que var no convierte a C# en un lenguaje de tipado dinámico: el tipo se determina una sola vez, en tiempo de compilación, a partir del valor asignado, y luego queda fijo igual que si se hubiera escrito explícitamente.

6. Tipos por Valor y por Referencia

Una variable de tipo por valor contiene directamente su dato. Al asignarla a otra variable, se copia el valor completo; ambas variables quedan totalmente independientes entre sí.

Una variable de tipo por referencia no contiene el dato directamente, sino una referencia (dirección) hacia un objeto almacenado en el heap. Al asignarla a otra variable, ambas terminan apuntando al mismo objeto.

Tipos por valor	Tipos por referencia
struct y todos los tipos numéricos bool, char, enum	class string array delegate object

```
int a = 5;
int b = a;    // se copia el valor
b = 10;      // a sigue siendo 5

int[] x = {1, 2, 3};
int[] y = x;  // y referencia el mismo arreglo que x
y[0] = 99;   // x[0] también cambia a 99
```

7. Operadores

Categoría	Operadores
Aritméticos	+ - * / %
Asignación	= += -= *= /= %=
Comparación	== != > < >= <=
Lógicos	&& !
Incremento/Decremento	++ --

Para los tipos de referencia, el operador `==` compara por defecto si dos variables apuntan al mismo objeto en memoria, no si su contenido es igual. `string` es una excepción notable: aunque es un tipo de referencia, `==` está sobrecargado para comparar el contenido de los caracteres.

8. Entrada y Salida por Consola

```
Console.WriteLine("Cual es tu nombre?");  
string nombre = Console.ReadLine();  
Console.WriteLine($"Hola, {nombre}");
```

`Console.ReadLine()` siempre devuelve un `string` (o `null` si no hay entrada). Para trabajar con números es necesario convertir el texto explícitamente:

```
string entrada = Console.ReadLine();  
int numero = int.Parse(entrada);  
  
// Alternativa segura, no lanza excepcion si falla  
bool exito = int.TryParse(entrada, out int resultado);
```

El símbolo `$` antes de una cadena permite insertar expresiones directamente dentro de `{ }`, evitando concatenaciones manuales con `+`.